

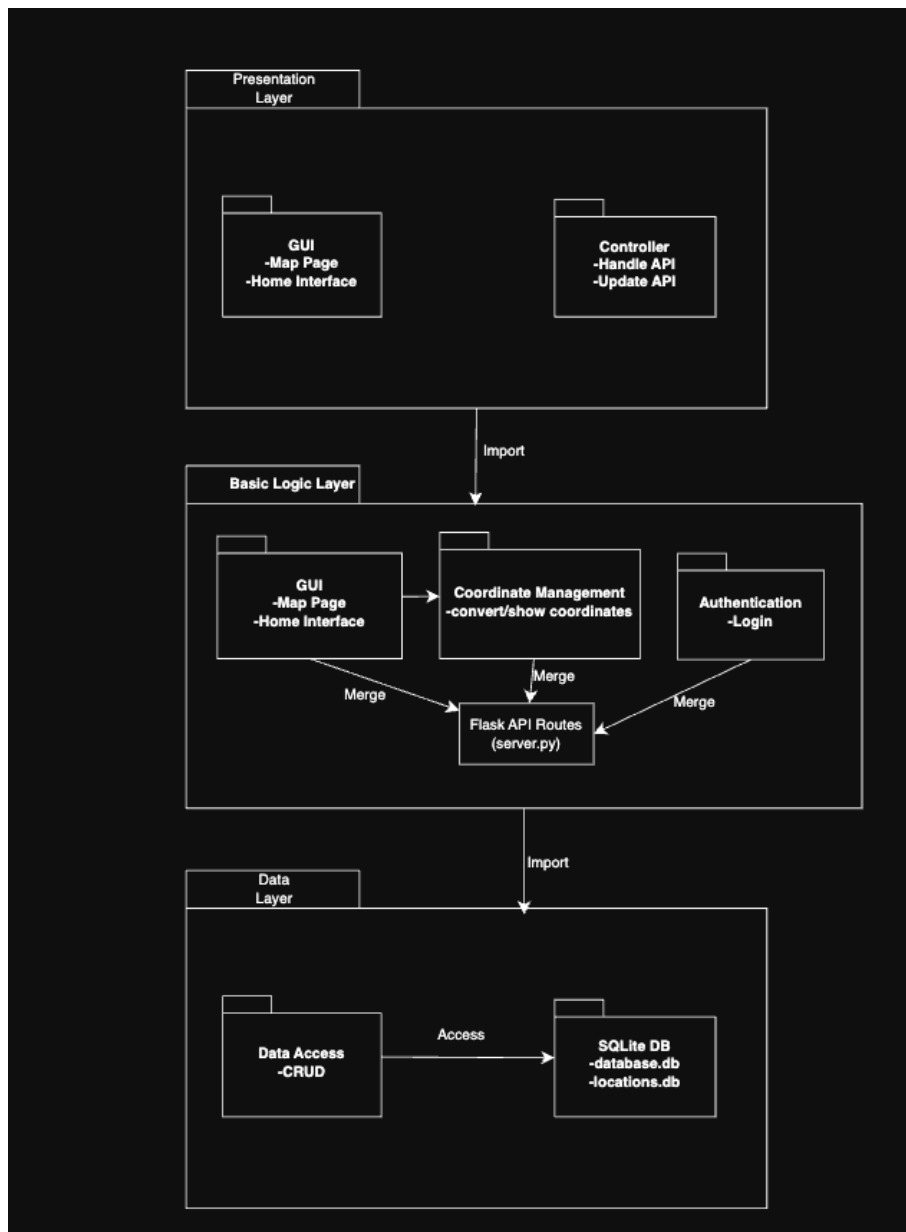
Project Overview

The fundamental issue at hand that our group aimed to address with our project was slow Wifi connection speeds on the UNC Charlotte campus. Many people throughout the course of Charlotte's recent history have complained about the wifi connection speeds on campus, so we aimed to design an application that can display the places on campus where the wifi is likely to be slow - a "Digital Weather" of sorts. Our goal is that users on the UNCC campus would be able to log in and see the places and times where people are on the wifi the most, and thus be able to anticipate in advance for slow wifi connection speeds. The hope here is that people could use the information obtained from our web service and plan where/when they use their technological devices on campus so that the on campus wifi routers are able to adequately keep up with all of the devices on the on-campus network. Because our application will allow for on campus users to be strategic with where/when they use the wifi, this will greatly alleviate unexpected wifi outages caused from too many users eating up all of the wifi available in an area.

Architectural Overview

We had originally favored a design that would passively track the data usage of anyone who came within the vicinity of campus through the use of an app, but this was scrapped due to security concerns. The intent was to have the users download an app, and once your device was registered onto the app it would track your location in the background. This had a number of design constraints that couldn't be adequately allocated for, as well as likely draining a lot of resources on the individual devices where it would run. It also fundamentally begged the question of when/where it would and wouldn't register devices, which would have been impossible to do accurately. Instead, we insisted that our application functions primarily through a Client-server architecture where the user interacts with a database server, providing it with login information, pin location/rating, reviews, and the server then updating the heatmap with the information obtained from the user. This new design allows us to store user information in a database on the cloud and to provide a stable answer for whose data would be recorded and where, allowing for a much more streamlined design.

Subsystem Architecture



The Presentation layer is responsible for everything the user sees and interacts with in the browser. The GUI components (e.g., Map page, forms, device lists) render the current state of the system, while the client-side controller logic takes user input, manages React state, and delivers REST calls to the backend. It never directly accesses the database or business rules; instead, it solely communicates with the API layer via HTTP, maintaining the user interface's independence from the computation or storage of data.

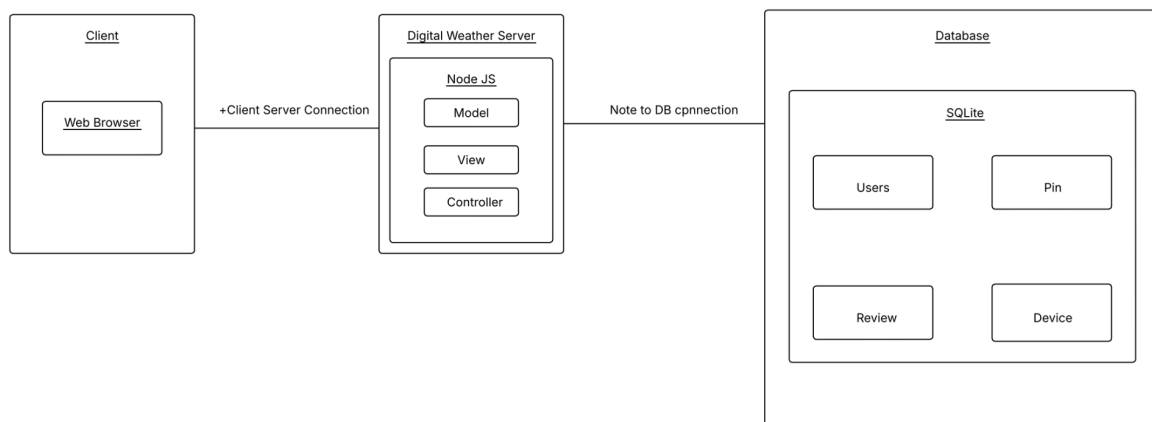
The Business Logic layer controls the main application behavior. Flask route handlers (in `server.py`) expose REST endpoints for devices, locations, coordinate conversion, and authentication. These routes coordinate calls to domain services like the Weather Device Logic (`device_generator.py`) and Coordinate Management (`coordinate_converter.py`), validate

requests, and enforce permission requirements. The Authentication/Infrastructure components (e.g., auth.py, extensions.py) cover cross-cutting concerns, including login processing, CORS settings, and error management.

Lastly, all vital data must be preserved and retrieved using the Data package. The Data Access section (e.g., models.py) works as an ORM/data gateway that allows CRUD operations over devices, locations, and any related entities. In order to avoid requiring upper levels to understand table schemas or query syntax, it conceals SQL/SQLite details behind higher-level functions. The database package (SQLite files such as database.db and locations.db) supplies the actual storage procedure, while the models mediate all access to it.

Deployment Architecture

This software will run on a single processor



We have a client side which is a react server located in a web browser. This connects to the local digital weather server that runs with Node JS. This is the server side in the backend, and includes all of our html pages, styling, and front end JS files. This then connects to our SQLite database, which has schemas for Users, Pins, Reviews, and Devices.

Persistent Data Storage

Our application requires a limited amount of user data to persist between alternative executions of the program, this being the user's username, password, the reviews and pins that they gave, and the device data generated to fill the heatmap. The user's information is placed into an Sql lite database controlled using flask sql alchemy. This allows us to ensure that each user is only counted once, providing more accurate results. Our project was set up this way to allow us to have stateless Client-Server communication, ensuring overall stability.

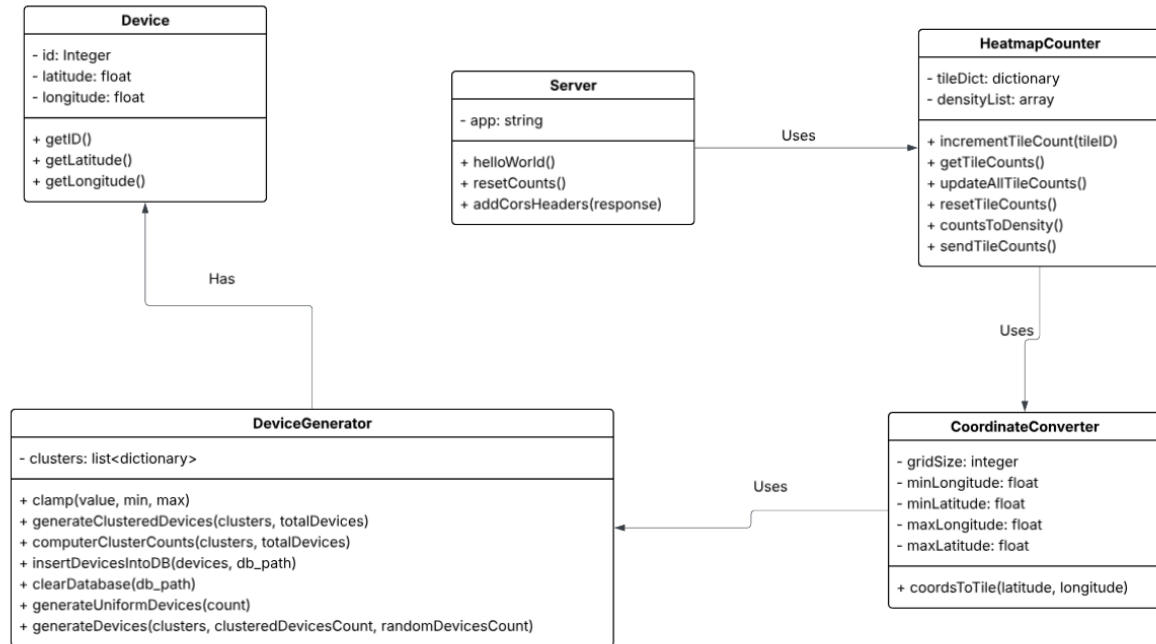
Global Control Flow

Our application is entirely procedural driven, meaning that logging in will provide the user with the information that they need to have for their visit to the site. Our system requires that the users provide their login credentials which will be stored and used for all in-app interactions. The interactions include putting pins on the map which allows the user to leave their reviews for the wifi accuracy in an area. The user may only comment or review on a pin that another user has left, but the pin that the user has made they are allowed to delete at will. These events are not time dependent, requiring only that the pins exist whenever the user interacts with them(or, the user can create a pin that can be interacted with at a later time). Multiple users are allowed to interact with the map at the same time, allowing for real-time communication and updates to occur. This only requires that each user login on their own separate device.

Detailed System Design

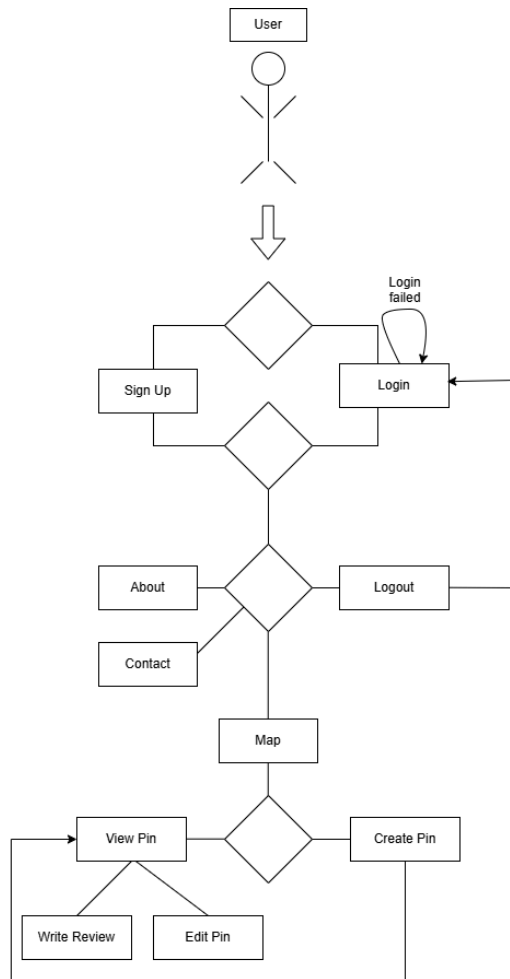
The design starts with the actual web server itself. The application hosts all of the generic templates that are required to build itself already at runtime, including the map of the campus, the background styling, and the heat map that will generate data as it is entered into the dataset. The user must then interact with the database in order to create their own login authentication, which either allows the user to enter with their previous username and password or set up a new username and password in which they may use to interact with the application in the future. The application then establishes a client-server connection in which it utilizes nodejs to specify the fields with the values that they will hold, as well as label what is going on in the actual web application. When the application is under its main course of action, it involves a constant stream of accepting the user's input in the form of pins. Each pin is inherently linked to a specific user(i.e. the user that created it), and the user can drop a review and/or comment on the pin that they have created. There is also an additional layer of functionality where the user can also comment on other users' pins and leave a review on their pins(each user gets 1 review per pin), but said user is not allowed to remove a pin made by the other user.

Static view



Our server has API endpoints to send back end data to the front end. The server sends a list of numbers that determines the color for each tile. The server calls the heatmap counter, which gets the list of numbers. It calculates this by converting the number of devices in each tile to a number which will represent its color on the map. The heatmap counter calls the coordinate converter to find out how many devices are in each tile. The coordinate converter does this by dividing the map into sections given its coordinate corners. Then it pulls each device, generated by the device generator, and plots its location into one of the sections on the map. It figures out which section, or tile, it belongs to, and adds a number to the dictionary counter the number of devices for that tile. The Device generator takes a list of dictionaries that give the coordinate location, name, radius, and weight that correspond to one location each on the map. It is given a number of devices to generate, and it generates them in those given areas.

Dynamic view



The user moves through the webpages and its related models as follows: First they are prompted to login, and if they have no account, they are prompted to register. Once they register or login, they are automatically redirected to the map page. With the navigation bar, the user can visit the about and contact pages, or logout if they choose to. At the map page, the user can either view an existing pin to leave a review on the location of the pin or delete/edit (granted they are the original creator of the pin) or create a new pin. If they posted a review, they can edit/delete it as well.